

BoardroomAI — Technical Overview

Architecture, algorithms, data model, testing and judge Q&A — Track 2, Multi-Agent Collaboration Systems.

Track 2 · Multi-Agent Collaboration Systems

1. What it is

A founder submits a pitch. Eight AI executives with distinct personalities, expertise and standing disagreements debate it live across three phases — opening statements, cross-examination, closing positions — then vote individually and produce a full investment report.

If you remember one sentence, make it this one:

The model does not run the meeting. Our orchestrator does. Turn-taking, topic detection, memory, quorum and vote tallying are deterministic TypeScript. The model is called once per turn, statelessly, and does not know the other seven executives exist.

That sentence is the whole architecture, and it is what separates this from a prompt with eight personas in it.

By the numbers

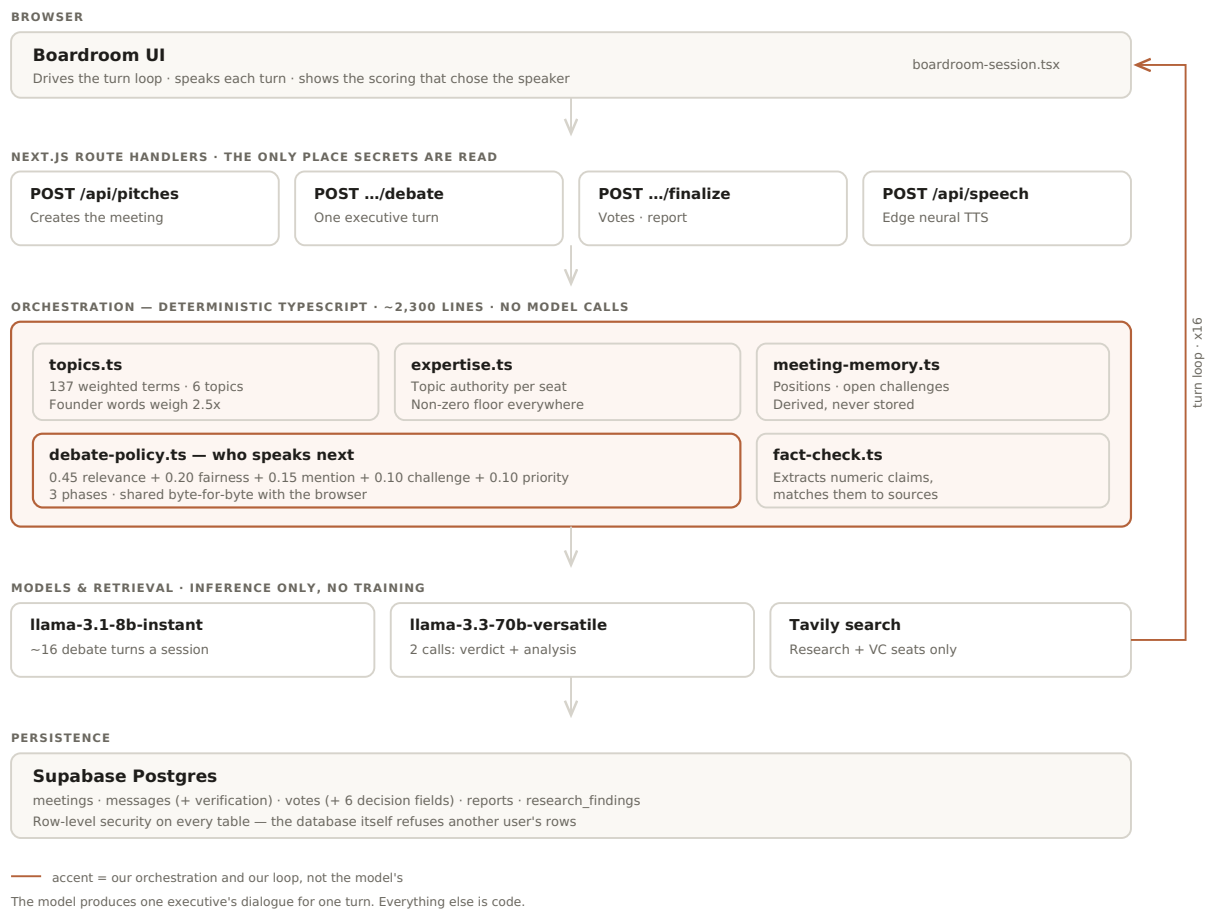
Application code	~14,500 lines across 186 TypeScript files
Orchestration layer	~2,300 lines, zero model calls
API route handlers	14
Database tables	8, all with row-level security
Model calls per session	~16 debate turns + 2 report calls
Deterministic test suites	5, all passing
Training / fine-tuning	None. Every persona is a prompt.

2. Stack, and why each piece

LAYER	CHOICE	REASON
Framework	Next.js 16, App Router	One codebase for UI and API; route handlers keep the Groq key server-side
Language	TypeScript, strict + <code>noUncheckedIndexedAccess</code>	The model returns untrusted JSON; types are the first defence
Database	Supabase (Postgres)	Managed Postgres, auth and row-level security in one
Inference	Groq	A session is 16 sequential calls a user watches happen; latency is the product
Search	Tavily	1,000 free searches/month for the evidence seats
Voice	Edge neural TTS → Web Speech fallback	Free, consistent across machines, degrades rather than fails

Nothing is trained. Groq is inference-only. All eight personalities live in `lib/ai/executives.ts` as prompt strings.

3. Architecture



One board session, end to end. Accent marks our orchestration and our turn loop — not the model's.

The invariant that holds it together. A component never fetches directly and never imports a Supabase client:

```
component → features/<x>/service.ts → /api/... → lib/server/... → Supabase / Groq
```

The invariant that makes the client trustworthy. `debate-policy.ts`, `topics.ts`, `expertise.ts` and `meeting-memory.ts` contain no server-only imports. The browser and the route handler run *the identical code* to decide who speaks next — not two implementations that agree by luck.

4. The core algorithm — speaker selection

The old rule was "whoever has spoken least". That is a rota, not a meeting: a founder could say "*our model cannot scale*" and the next speaker would be whoever happened to be up.

Selection is now a weighted score computed for every eligible executive:

```

score = 0.45 · relevance
       + 0.20 · fairness
       + 0.15 · founderMention
       + 0.10 · disagreement
       + 0.10 · priority
       - 0.30 · recencyPenalty

```

The five weights sum to 1, so a score reads as 0–1. The recency penalty sits outside that sum because it is a damper, not a signal.

4.1 relevance — 0.45, the dominant term

```

authority = expertise[executiveId].topics[topic]    // 0–1, hand-tuned
relevance = topic === "general"
           ? 0.5
           : 0.5 + (authority - 0.5) × max(0.35, topicConfidence)

```

Two design decisions worth defending:

Every executive has a non-zero floor on every topic. A board member with literally nothing to say about burn rate is not a board member. The floor is what lets a CMO push back on a CFO's assumption instead of being silently disqualified from the argument.

Relevance is damped by topic confidence. A weak topic read should not override fairness. When the detector is unsure, relevance collapses toward 0.5 and stops discriminating — which is the honest behaviour.

4.2 fairness — 0.20

```

fairness = 1 - (turnsSpoken / maxTurnsAnyoneHasSpoken)

```

Measured across the whole session, not the phase, so an executive who was quiet in round one is pulled forward in round two.

Fairness is not what guarantees everyone speaks — nothing in the score is. Eligibility filtering does that structurally, which is why there is no starvation bonus. See §4.6.

4.3 founderMention — 0.15

Being named is close to being handed the floor. Matched against the executive's id, first name, surname, role, role-without-"Agent", and hand-written aliases (`counsel`, `finance`, `tech lead`). Decays 0.35 per message as the conversation moves past it, over a 3-message window.

4.4 disagreement — 0.10

Fires when a colleague named this executive *and* used a challenge marker (`disagree`, `that assumes`, `why do you`, `justify`, ...) **since they last spoke**. A board where challenges go unanswered is a board reading statements aloud.

4.5 recencyPenalty — subtracted

1.0 if they spoke last turn, 0.4 if two turns ago. Nobody speaks twice in a row.

4.6 Why there is no starvation guard

An earlier version added +10 to anyone still "owed" a turn, on the theory that a soft fairness weight could not guarantee everyone speaks.

It was **wrong twice**, and removing it is a better answer than tuning it. It could not fire selectively — eligibility already excludes anyone at their allowance, so with one turn per executive per phase the "owed" set always equalled the eligible set, and every score was inflated by 10. The UI showed `10.67, 10.46, 10.44` instead of `0.67, 0.46, 0.44`, which made the ranking unreadable.

And it was unnecessary. A phase ends only once the transcript holds `seats × allowance` executive turns, and only under-allowance executives can contribute them. **Eligibility filtering is the guarantee**; no bonus required.

This was caught by computing the worked example below rather than assuming it.

4.7 Worked example — real output from the scorer

Founder says "Our AI model cannot scale." — empty transcript, so fairness is tied at 1.00 and relevance is the only differentiator:

EXECUTIVE	RELEVANCE	FAIRNESS	PRIORITY	SCORE
Priya Nair — CTO	0.92	1.00	0.60	0.67
Elena Vasquez — CEO	0.38	1.00	0.90	0.46
Nadia Petrov — Research	0.42	1.00	0.55	0.44

Note the CEO has the *highest* seat priority (0.90) and still loses by 0.21 — relevance at 0.45 weight dominates a 0.10 tiebreak, which is the intended behaviour.

Same board, different sentence — "Our CAC is too high and the burn rate is climbing":

EXECUTIVE	RELEVANCE	FAIRNESS	PRIORITY	SCORE
Marcus Webb — CFO	1.00	1.00	0.65	0.72
Jonah Kessler — VC	0.80	1.00	0.85	0.65
Elena Vasquez — CEO	0.45	1.00	0.90	0.49

Nothing changed but the founder's words, and the floor moved from engineering to finance.

5. Topic detection

Read before every turn, so it must be cheap. A model call here would add one request per turn — doubling a session's cost and latency to answer a question a weighted dictionary answers well enough.

137 weighted terms across 6 topics (technical 29, financial 25, market 22, growth 23, legal 21, strategy 17). Multi-word phrases score higher because they are less ambiguous: "*unit economics*" is unmistakably financial, while "*cost*" appears in a sentence about latency just as often.

```
score[topic] += termWeight
    × RECENCY_DECAY^age           // 0.65 per message back
    × (fromFounder ? 2.5 : 1)     // the founder redirects the room
    × (1 + log2(hits))            // diminishing returns
```

Three subtleties, each of which was a bug first:

Whole-word matching. `cac` must not fire inside `cache`.

Longest-first with masking. `"single thesis"` (weight 4) and `"thesis"` (weight 3) both matched the same four words, contributing 7 to *strategy* — enough to beat a real technical signal. Terms are now sorted longest-first and matched spans are consumed, so the specific phrase wins and the generic word inside it never fires.

The founder weighs 2.5×. At parity, a CEO asking for the "single thesis" out-scored the founder admitting the model cannot scale, and the floor went to the VC instead of the CTO. The founder's words are new information, not commentary.

A topic must also beat the runner-up by **1.25×** to be called at all; otherwise it reads `general` and selection falls back to fairness and priority. Without that, one stray word decides the topic and speaker selection jitters turn to turn.

6. Executive memory

Each executive carries, per meeting:

- **positionsTaken** — their own prior points, capped at 4
- **openChallenges** — challenges aimed at them they have not answered
- **unansweredQuestions** — questions they asked the founder ignored
- **topicsCovered** — subjects they have already spoken to

Memory is derived from the transcript, never stored. The transcript is already the single source of truth for a session; a parallel memory table could only ever disagree with it, and would need writing on every turn.

Extraction is deterministic string work, not a summarisation call. Memory is rebuilt before every turn — an LLM pass would add a request per turn purely to restate text we already hold. A "position" is the longest declarative sentence over 40 characters that is not a

filler opener; a challenge is a message naming a colleague *and* carrying a disagreement marker, still open only if the target has not spoken since.

Caps exist because the whole memory block rides in the system prompt on every turn. Unbounded, a long session spends more of its token budget reminding an executive what they said than letting them say anything new.

7. Three-phase debate

PHASE	WHAT THE PROMPT DEMANDS
Opening	First read from your own seat; raise the thing that most concerns you
Cross-examination	Do not restate your opening. Put a direct question to a named colleague whose reasoning you doubt, or answer one put to you
Closing	Where you landed, what changed your mind, the single condition that would move you

Phase is **derived from the transcript**, not stored. Derived state cannot disagree with the transcript, which matters because client and server both compute it. A `phase` column would need writing every turn and would be wrong the moment a write failed.

Profiles are env-configurable — session length is a demo constraint as much as a product one:

PROFILE	TURNS/EXEC	TOTAL (8 SEATS)
<code>full</code>	1 / 1 / 1	24
<code>demo</code>	1 / 1 / 0	16
<code>deep</code>	2 / 2 / 1	40

Output budget varies by phase — 180 / 260 / 200 tokens. Cross-examination legitimately needs more room: naming whose reasoning you doubt, saying what is wrong with it, and putting a question does not fit in a truncated clause.

8. Prompt construction

The old prompt was persona + startup facts + "don't repeat anyone". That produced eight people reading prepared statements into the same room, because nothing gave them anything specific to react *to*.

Each turn now assembles:

1. Persona (character, and **who they habitually clash with**)
2. That executive's objective for the meeting
3. Company facts and the founder's pitch
4. **Current phase** and what it demands

5. **Current topic**
6. Who else is at the table, by name
7. **Claims on the table, attributed** — "Marcus Webb (CFO) argued: ..."
8. **Their own prior points** — do not repeat these
9. **Open challenges against them** — answer this first
10. Unanswered founder questions
11. **Verified external data**, held strictly separate from reasoning
12. Speaking rules

The "who they clash with" line in each persona is load-bearing. Eight reasonable people asked to evaluate a pitch will converge; eight people with declared standing tensions will not. **The vote split the board produces is downstream of those sentences.**

The trick that makes eight people from one model

In `lib/ai/client.ts`, the founder's messages map to `role: "user"` and *every executive's* messages — including other executives' — map to `role: "assistant"`. From the model's point of view the entire board transcript is its own prior output. It is one model wearing a different personality card each turn.

That is why colleagues' lines are prefixed `"Marcus Webb (CFO Agent): ..."` and why the prompt says *never speak for another executive* — without both, the model writes the rest of the meeting in one response.

Context windowing — a real optimisation

`TRANSCRIPT_WINDOW = 6`. Sending full history made cost quadratic: with 8 executives over 2 rounds the 16th speaker re-sent the 15 turns before it, ~120 message transmissions per session instead of 16. The founder's most recent message is always kept even if it fell outside the window — it is the one turn the board is actually answering.

9. Research grounding and fact-checking

Retrieval

Fires only for **evidence seats** (Research, VC) on **evidence topics** (market, financial). One basic-depth Tavily query, top 3 results above a 0.4 relevance floor, 6-second timeout.

Deliberately a pre-fetch, not model-driven tool calling. Debate turns run on an 8B model; small models emit malformed tool calls often enough that every turn needs a repair path, and a failed call mid-debate is a dead turn the founder watches happen. The orchestrator decides deterministically instead: this seat, this topic, gets evidence. Same outcome, no parsing risk, decision inspectable in code. That is a deliberate trade of flexibility for reliability.

Findings persist to `research_findings` keyed by meeting. They used to live in a module-level `Map` — which worked on a dev server and **silently broke on serverless**, because `finalize` can

execute in a different instance from the debate turns, so the report read an empty cache and citations vanished on the deployed URL only.

Verification

Injecting evidence and trusting the model to honour it is not a check. After each evidence-seat turn:

```
extract numeric claims → match against retrieved snippets → supported / unsupported
```

Claim patterns are narrow on purpose: currency with magnitude, bare magnitudes, percentages, multiples. Every loosening turns ordinary prose into a "claim" and buries the real unsupported figures in noise — "*under 3 sentences*" must never be flagged as an unverified statistic.

Hedged figures are excluded. "*I'd estimate around \$2 billion*" is exactly the honest behaviour the prompt asked for, so flagging it would punish compliance.

No evidence means no verdict, not a blanket accusation. `checked: false` is meaningfully different from "checked and clean".

Unsupported figures render a warning badge on the transcript turn. Report citations bypass the model entirely — it is never asked to produce a URL, so a citation cannot be hallucinated.

10. Voting and the report

Per-executive votes

Six fields beyond yes/no/conditional: **confidence (0-100), biggest risk, biggest strength, required milestone, cheque size, return horizon**. Eight executives × six fields = 48 data points per session, all rendered on the report.

The header line reads "*3 yes · 3 conditional · 1 no — average confidence 62. The board did not reach agreement, which is itself a finding.*" A split board that is individually confident means something very different from a split board that is collectively unsure, and the headline score cannot distinguish them.

Two calls, not one

Asking one call for the verdict, eight six-field votes *and* the consensus map reliably blew the output budget — the model truncated mid-array and JSON parsing threw, losing the entire write-up. Two schema-constrained calls each fit comfortably, and the second is allowed to fail without taking the report with it.

Structured output, three layers of defence

1. `response_format: json_schema` with `strict: true` — the model validates its own output
2. Only some Groq models accept that; the rest reject with a **400 specifically**, which triggers a fallback to JSON mode with the schema in the prompt. Any other status is a real error, not a capability gap

3. `normalise()` clamps numbers, coerces enums, caps array lengths

Layer 3 is load-bearing: the report UI indexes into lookup tables by SWOT title and risk level, so an off-schema string would **crash the page**, not degrade.

One detail worth knowing: the vote schema is **built per request** so `executiveId` is an enum of the ids actually seated. As a free string the model reliably answers with the executive's *name*, which no lookup matches.

11. Voice

Two providers behind one seam:

1. **Edge neural TTS** via `/api/speech` — eight distinct voices including Indian and British English where the persona names suggest it. Free, and the same voice id sounds identical on every machine
2. **Web Speech API** — robotic and OS-dependent, but cannot fail in a way that leaves the board silent

Edge's endpoint is not a published API, so failure is an *expected branch*. One failure disables Edge for the session — retrying a 12-second timeout on every turn would be worse than the robotic voice.

The architectural point: speech runs *inside* the rate-limit wait, not after it. Groq's free tier forces ~15s between turns; that dead air is now the executive talking. A turn takes whichever is longer.

12. Data model

TABLE	HOLDS
<code>profiles</code>	User profile, auto-created by a Postgres trigger on signup
<code>meetings</code>	One session: startup, pitch, stage, status
<code>meeting_executives</code>	Which executives are seated, in what order
<code>messages</code>	Every transcript line, plus its fact-check result
<code>votes</code>	One row per executive: vote + six decision fields
<code>reports</code>	Score, verdict, SWOT, risks, consensus map, confidence breakdown
<code>research_findings</code>	Evidence actually retrieved, per meeting
<code>workspace_data</code>	The five studio deliverables
<code>activity_events</code>	Audit feed

Every table has row-level security keyed on `auth.uid()`:

```
create policy "Users manage their meetings" on public.meetings
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());
```

Postgres itself refuses to return another user's rows — this is not enforced by application code. If a route handler forgot to filter by user, the database would still return nothing. That is why the Supabase anon key is safe in the browser. Child tables inherit ownership via an `exists` subquery back to `meetings`, so there is one source of truth for who owns a session.

13. Reliability engineering

Rate limits are absorbed, not surfaced. Groq reports retry timing in two places and neither is guaranteed: a `retry-after` header, and a sentence inside the error body ("*Please try again in 340ms*"). The client reads the header, falls back to parsing the sentence, then backs off exponentially — 3 retries, capped at 10s. A session should not die on a 340ms delay.

Finalize is idempotent. A session with a report returns it rather than regenerating; a double-click cannot bill a second generation.

Failure isolation. Report and votes are written *first*, then deliverables are attempted in a `try/catch` that swallows failure. The report is the contract; the studio artefacts are a bonus.

The founder's message persists before the model is called, so their turn survives a reload even if the model call then fails.

Schema drift does not take down the app. Meeting queries try the full column set, detect a missing-column error specifically, and retry without it. A database one migration behind loses fact-check badges — not the boardroom.

14. Testing and evaluation

Five deterministic suites, no network, no database, no model — the whole orchestration layer is pure functions precisely so it can be tested this way.

SUITE	COVERS
<code>test:orchestration</code>	Topic detection, scoring, fairness, starvation, memory
<code>test:session</code>	Full 24-turn simulation and its structural guarantees
<code>test:factcheck</code>	Claim extraction, false-positive resistance, hedge handling
<code>test:tts</code>	SSML well-formedness, frame parsing, voice assignment
<code>test:metrics</code>	Rank correlation, tier separation, deliberation rate

Bugs these caught before they shipped: female executives assigned male voices once the female pool ran out; `$1.2M` read aloud as "1.2 dollarsM"; `Path:audio.metadata` matching `Path:audio` and corrupting the MP3 stream; `$4.5 billion` double-counted as two claims; the VC beating the CTO on scaling questions.

The eval harness

`npm run eval` runs six hand-graded pitches (strong / mediocre / weak) through the real pipeline and reports:

- **Tier separation** — did every strong pitch outscore every mediocre one?
- **Rank correlation** — Spearman against the grading
- **Deliberation rate** — % of sessions where the board did *not* vote unanimously
- **Score spread** — variance across repeat runs

Deliberation rate is the one that tests the architecture rather than the judgement. A rate near 0% would mean eight personas collapsing into one voice — precisely the failure the multi-agent design exists to prevent.

State this honestly: the fixtures are graded against investor criteria, not observed outcomes. The eval measures whether the board discriminates *consistently in the direction a human would*. It does **not** measure whether the board predicts which startups succeed. Claiming predictive validity from six written pitches would be the same unearned confidence this project criticises in its own Research agent.

15. Judge Q&A

"Isn't this a ChatGPT wrapper?" A wrapper passes text to a model and shows the reply. The model here never decides who speaks, when the debate ends, or how votes are tallied — that is ~2,300 lines of deterministic code. It produces one executive's dialogue for one turn and does not know the other seven exist. Take the model out and the meeting logic still runs; take the orchestrator out and you have eight disconnected chatbots.

"Couldn't one prompt give you eight perspectives?" No, and this is the interesting part. A single prompt generates all eight opinions in one pass, each conditioned on the ones it just wrote — which collapses toward internal consistency. You get eight paraphrases of one position. Here each turn is a *separate inference call* where the agent sees colleagues' actual words and is instructed to challenge rather than echo. The vote split is the evidence that produces genuine divergence.

"Did you train the model?" No. Groq is inference-only. Fine-tuning teaches form, not facts — we would need thousands of board transcripts and would still have a model that talks like a board member while knowing nothing about the startup in front of it. The leverage is in orchestration and retrieval.

"What about hallucination?" Three constraints — schema enums, per-request id enums, normalisers — plus a verification pass that flags numeric claims absent from retrieved sources. But be precise: **we constrain shape and check numbers. We do not verify prose.**

"Is the data secure?" The Groq key is read only inside route handlers. Every table has RLS keyed on `auth.uid()`, so Postgres refuses cross-user reads regardless of application bugs.

"What breaks first at scale?" Groq per-minute token budgets, then the client-driven loop (closing the tab stops the session — fine for a live meeting, wrong for scale; the fix is a server-side queue), then the 120s finalize duration.

"Why does the client drive the debate?" Deliberate. A server loop is simpler, but the founder would watch a spinner for four minutes then receive a wall of text. Turn-by-turn makes it a meeting they attend rather than a report they wait for.

"How do you know the score is any good?" Run `npm run eval`. And say plainly what it does and does not show — §14.

16. Known limitations

Stated because a team that knows its gaps is more credible than one that oversells.

- **Topic detection is keyword-based.** *"Our AI model cannot scale"* works; *"we can't handle more users"* has no keyword and reads `general`.
- **Verification covers numbers only.** A misleading qualitative claim passes.
- **Research is narrow.** Two of eight seats, two of six topics. The CFO — who discusses money constantly — is not an evidence seat.
- **No predictive validation.** The eval measures consistency, not prediction.
- **Edge TTS depends on an unpublished endpoint.** Mitigated by the fallback, not eliminated.
- **Late-session executives cannot see the opening.** A 6-turn window plus 4 remembered positions means round-one detail is partly out of view by the closing round.

Each has a known next step. "We know, and here is the fix" beats "we hadn't considered that."

17. Demo checklist

- Set `NEXT_PUBLIC_DEBATE_PROFILE=demo` — 16 turns instead of 24
- Pre-run a completed session in a second tab; a live session takes ~4 minutes
- Test audio on the presentation machine; know where the mute button is
- Have `debate-policy.ts` open in the editor — 40 lines of real code beats describing it
- Run `npm test` in front of a judge who asks how you know it works
- Know three numbers cold: **8 personas, 2 models, 0.45 relevance weight**

The line to lead with:

The model doesn't run the meeting — our orchestrator does. Turn-taking, quorum and vote tallying are deterministic code; the model is called once per turn and doesn't even know the other seven executives exist.